

¿Qué es JSON y para qué sirve?

JSON (JavaScript Object Notation) es un formato de texto ligero, independiente del lenguaje, diseñado para el almacenamiento y transporte de datos. Su diseño se basa en un principio fundamental: debe ser fácil de leer y escribir para los humanos, y rápido de procesar (parsear) y generar para las máquinas.

JSON cumple roles críticos:

- **Comunicación en la Web:** Es el estándar de facto para el intercambio de cargas útiles (payloads) en servicios web, especialmente en APIs RESTful y GraphQL.
- **Archivos de Configuración:** Herramientas de desarrollo, entornos y aplicaciones lo utilizan para definir sus parámetros (ej. package.json, configuraciones de VS Code).
- **Almacenamiento NoSQL:** Motores de bases de datos documentales como MongoDB, CouchDB o Firebase almacenan sus registros utilizando JSON (o variantes binarias compatibles).

¿Cómo es el formato de JSON?

A nivel sintáctico, JSON se construye sobre dos estructuras de datos universales que mapean directamente con casi cualquier lenguaje de programación:

Objeto: Una colección desordenada de pares clave/valor. Comienza con { y termina con }. Cada clave es seguida por dos puntos : y los pares se separan por comas .

Arreglo (Array): Una lista ordenada de valores. Comienza con [y termina con]. Los valores se separan por comas ,.

Reglas estrictas de sintaxis:

- Las claves siempre deben ser cadenas de texto encerradas entre comillas dobles (nunca simples).
- Los valores permitidos se limitan a: String (entre comillas dobles), Number (entero o punto flotante, sin comillas), Boolean (true o false), Null (null), Objeto (anidado) o Array.
- No admite comentarios.

Dar un ejemplo de un objeto y de una colección de Objetos en Java y su equivalente en JSON.

Representación de objetos en Java.

```
public class Empleado {  
    public String nombre;  
    public int edad;  
    public boolean activo;  
  
    public Empleado(String nombre, int edad, boolean activo) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.activo = activo;  
    }  
}
```

// Creación del objeto en memoria

```
Empleado empleado = new Empleado("Elena", 28, true);
```

Equivalente en JSON:

```
{  
  "nombre": "Elena",  
  "edad": 28,  
  "activo": true  
}
```

Representación de colección de objetos en Java:

```
List<Empleado> equipo = new ArrayList<>();  
equipo.add(new Empleado("Elena", 28, true));  
equipo.add(new Empleado("Marcos", 35, false));
```

Equivalente en JSON:

```
[  
  {  
    "nombre": "Elena",  
    "edad": 28,  
    "activo": true  
  },  
  {  
    "nombre": "Marcos",  
    "edad": 35,  
    "activo": false  
  }  
]
```

¿Cuáles alternativas existen además de JSON?

Aunque el formato JSON es el más utilizado en el mercado actualmente, existen algunas alternativas como por ejemplo:

- **XML**: Basado en etiquetas <tag>. Es mucho más verboso que JSON, pero permite validaciones estrictas de esquemas mediante XSD. Sigue siendo estándar en sistemas corporativos legados, facturación electrónica y servicios SOAP.
- **BSON**: Extensión binaria de JSON. Es más rápido de escanear y añade soporte nativo para tipos de datos complejos como fechas (Date) o bytes crudos (BinData). Es el formato de almacenamiento interno de MongoDB.